Sistema generador de números pseudo aleatorios usando algoritmos de ordenamiento Big-O

Luna Leyva Azul Desiree, Ramos Ramírez Alberto Carlos, Santana Martínez Josué Eleazar
Universidad Nacional Autónoma de México
Facultad de Estudios Superiores
16 de marzo de 2026

***Resumen*—Los algoritmos de ordenamiento son procedimientos lógicos para organizar un conjunto de datos en un orden específico (ascendente o descendente). En ciencias de la computación, su eficiencia se mide con la notación Big-O, que describe cómo crece el tiempo de ejecución a medida que aumenta el número de datos (n).**

I. Introducción

La programación en lenguaje ensamblador permite una interacción directa con la arquitectura del CPU, facilitando el control preciso sobre módulos como la ALU, los registros de propósito general y la memoria SRAM. En el contexto de los sistemas embebidos, la eficiencia en el manejo de datos es crítica debido a las limitaciones de recursos de hardware.

El objetivo principal de esta práctica es desarrollar capacidades para la escritura de código de bajo nivel que manipule estructuras de datos en memoria. Para ello se utilizan punteros de memoria y ciclos de control que permiten transformar una tabla de números desordenados en listas organizadas bajo criterios específicos.

II. Teoría e importancia de los algoritmos de ordenamiento

El ordenamiento de datos consiste en organizar un conjunto de elementos siguiendo una relación de orden (numérica o alfabética). Su importancia radica en la optimización de otros procesos: un conjunto de datos ordenado reduce drásticamente el tiempo necesario para realizar búsquedas, permitiendo el uso de búsqueda binaria y facilitando la identificación de patrones o duplicados.

En sistemas con microcontroladores elegir el algoritmo correcto es vital, ya que algunos priorizan el ahorro de memoria RAM como Bubble Sort, mientras que otros buscan reducir la cantidad de ciclos de instrucción del procesador.

III. Notación Big-O

La notación Big-O es una herramienta fundamental en la programación para medir la eficiencia de nuestro algoritmo. Esto gracias a la evaluación del tiempo y el uso de memoria en función del tamaño de entrada, permitiendo al usuario comparar algoritmos y ayudando con la optimización del rendimiento del código.

Además de ayudar a describir el peor caso posible del algoritmo, es decir, el número máximo de operaciones que es posible o que debe realizar. Es muy importante considerar que no mide el tiempo exacto en segundos, más bien cómo crece el número de operaciones que realizará a medida que aumenta el tamaño de la entrada.

Cuadro I
Comparación entre algoritmos

Algoritmo	Complejidad	Comparaciones	Ciclos de reloj	Tiempo (μs)
Bubble Sort	$O(n^2)$	4950	112540	112538
Selection Sort	$O(n^2)$	4950	44383	43418

IV. Resultados

Cuadro II
Resultados experimentales

Parámetro	Bubble Sort	Selection Sort	Observaciones
Complejidad	$O(n^2)$	$O(n^2)$	Ambos cuadráticos
Comparaciones	4950	4950	$2n(n-1)$
Ciclos de reloj	113505	44383	Selection más rápido
Tiempo (μs)	113505	44383	Frecuencia 1 MHz

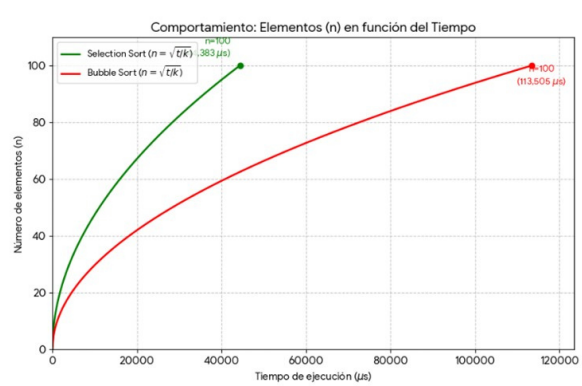

Figura 1. Gráfica Bubble vs Selection

V. Pseudocódigo

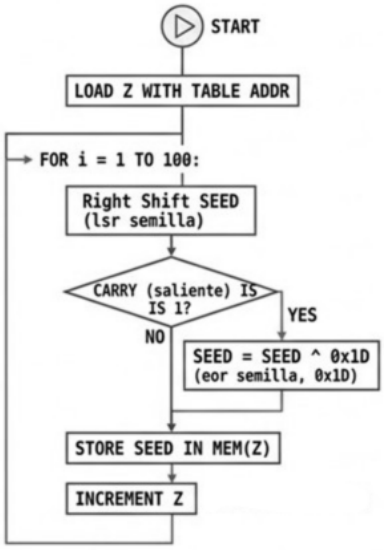

Figura 2. Generador de números aleatorios

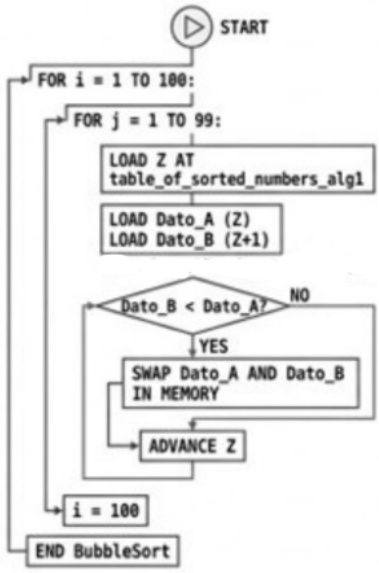

Figura 3. Implementación Bubble Sort

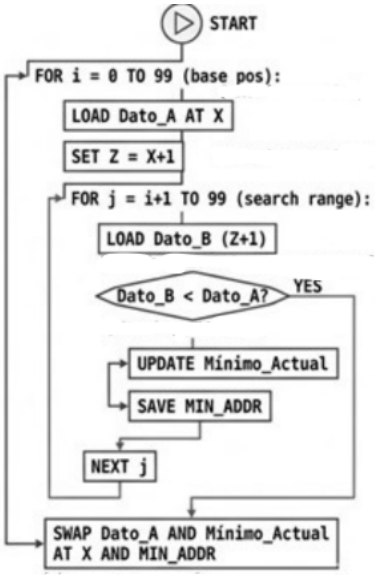

Figura 4. Implementación Selection Sort

VI. Análisis de resultados

Aunque la notación Big-O es la misma para ambos algoritmos ($O(n^2)$), el Selection Sort terminó la tarea en menos de la mitad del tiempo. Esto ocurre porque Bubble Sort realiza intercambios de memoria constantes dentro del ciclo interno, lo que incrementa considerablemente el número de ciclos de instrucción ejecutados.

La diferencia en la gráfica subraya la importancia de minimizar las instrucciones de almacenamiento (ST) en microcontroladores, ya que el Selection Sort solo realiza un intercambio por cada pasada externa, mientras que el Bubble Sort puede realizar cientos en el peor de los casos.

Big-O nos da la tendencia de crecimiento (la "forma"de la curva de tiempo), pero los detalles de implementación en Assembly (como el número de accesos a memoria RAM) son los que determinan el tiempo exacto final.

VII. Conclusiones

Luna Leyva Azul Desireé

Para esta actividad fui capacez de entender cómo es que funcionan los módulos de la EEPROM y RAM, y de esta forma comprender de manera práctica como es que se gestionan los datos a nivel de hardware haciendo uso de la ATMEGA 328p en lenguaje ensamblador

Ramos Ramírez Alberto Carlos

Para el diseño de sistemas embebidos que se puede pensar que son limitados por los resultados obtenidos, el Selection Sort es preferible sobre el Bubble Sort. A pesar de compartir la misma complejidad teórica, la reducción en las operaciones de escritura en memoria lo convierte en una solución más robusta y rápida para el procesamiento de datos internos.

Santana Martínez Josué Eleazar

Aunque ambos algoritmos poseen la misma complejidad teórica $O(n^2)$, las pruebas en el depurador demostraron que Selection Sort es significativamente más rápido. Elegimos

estos dos algoritmos por el tiempo de ejecución, si se hubieran utilizado otros, estaríamos habalndo de un retardo mayor en la ejecución del programa.

REFERENCIAS

[1] M, G. P. (2023, 21 abril). Explicación de la notación Big O con Ejemplos. freeCodeCamp.org. https://www.freecodecamp.org/espanol/news/explicacion-de-la-notacion-big-o-con-ejemplo/

[2] Correa, J. (2024, 5 agosto). Big O notation o notación Big O: Todo lo que necesitas saber 2025. Developero. https://developero.io/blog/big-o

[3] Elbloddeldev. (2026, 8 marzo). Big O Notation: Qué es y por qué es importante. El Blog del Programador. https://elblogdelprogramador.com/posts/big-o-notation-que-es-y-por-que-es-importante/